Vyapar SchemeHub

AI-Powered MSME Government Scheme Discovery, Verification & Recommendation Engine

Document: Complete Architecture & Implementation Manual

Version: 2.0 (Resilient Production Release)

Stack: React 19 + FastAPI + Supabase + Qdrant + Google Gemini

Table of Contents

1. **Executive Summary & Business Objectives**
2. **Complete System Architecture & High-Level Dataflow**
3. **Database Architecture & Relational Schema**
4. **Backend Deep Dive (FastAPI Services & APIs)**
    - Configuration & Environment Architecture
    - Server Lifespan & Initialization
    - API Endpoints & Routing Surface
    - Eligibility & Rules Evaluation Engine
    - Recommendation Engine & Scoring
    - RAG Vector Search & Google Gemini AI Chatbot
    - OCR Document Verification Pipeline
    - Ingestion & Vectorization Pipeline
5. **Frontend Deep Dive (React 19 + Vite Application)**
    - Application Entry Point & Architecture
    - Authentication & Token State Management
    - Component Hierarchy & Navigation
    - All Pages & User Flows
    - AI Scheme Assistant Chatbot Component
6. **Data Ingestion & Dataset Tools**
7. **Recent Resiliency Fixes & Architectural Upgrades**
8. **Setup, Execution & Operations Guide**

---

1. Executive Summary & Business Objectives

India has over 63 million Micro, Small, and Medium Enterprises (MSMEs) contributing nearly 30% of the country's GDP. The Central and State Governments offer hundreds of credit-linked subsidies, technology upgradation schemes, collateral-free credit guarantees, and export promotion incentives (e.g., CGTMSE, PMEGP, CLCSS, ZED Certification).

However, MSMEs struggle with:

- **Information Asymmetry:** Complex guidelines spanning hundreds of pages across different ministries.
- **Eligibility Uncertainty:** Inability to decipher if their business size, turnover, sector, or social category qualifies.
- **Documentation Burdens:** Confusing required documents leading to high application rejection rates.

Vyapar SchemeHub solves this by introducing:

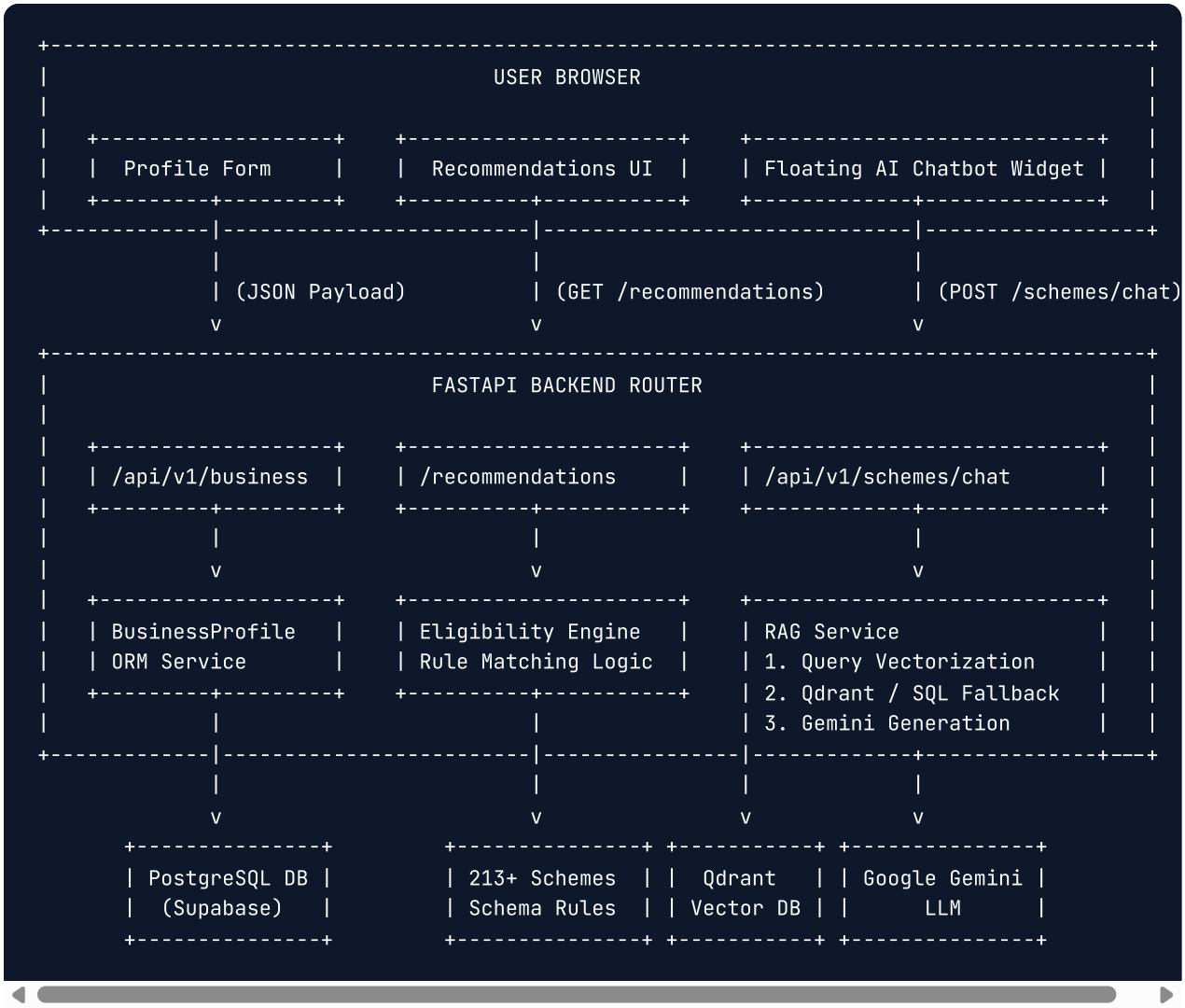
1. **Intelligent Recommendation Engine:** Rule-based matching engine that cross-references 213+ government schemes with business profiles.
2. **Explainable AI (XAI):** Detailed justifications of eligibility and non-eligibility breakdowns (e.g., exactly why a business was rejected based on turnover or location).
3. **RAG-Powered AI Scheme Assistant:** Real-time chatbot utilizing Google Gemini and high-dimensional semantic search over government scheme documents with an automatic SQL fallback.
4. **Automated Document OCR:** Computer vision verification extracting Udyam numbers, PAN, GST, and turnover directly from official certificates.

2. Complete System Architecture & High-Level Dataflow

> **Architecture Summary**
>
> The system is split into two primary tiers: a Vite-powered React Single Page Application (SPA) communicating over RESTful JSON with a high-performance Python FastAPI backend, supported by Supabase PostgreSQL, Qdrant Vector Cloud, and Google Gemini Generative AI.

Layer	Technologies	Key Responsibility
Frontend Client	React 19, Vite, Tailwind CSS, Lucide React, Axios	Responsive UI, profile creation form, matched scheme visualization, document upload interface, and conversational AI assistant widget.
API Gateway / Backend	FastAPI, Uvicorn, Pydantic v2, Python 3.10+	CORS negotiation, JWT authentication, rule verification, API endpoints for profiles, schemes, recommendations, chat, and OCR.
Relational Database	PostgreSQL (Supabase) / SQLite fallback (SQLAlchemy ORM)	Stores user records, 213+ structured scheme rules, MSME profiles, user recommendations, and OCR verification logs.
Vector Database	Qdrant Cloud / Local Vector Engine (768-dim embeddings)	Stores semantic chunks of government policy guidelines and PDF circulars for semantic retrieval.
Artificial Intelligence	Google Gemini 1.5 / 2.0 Flash / Pro APIs	Synthesizes RAG contexts into clear, structured natural language answers for general and scheme-specific user queries.
Document Intelligence	PyPDF2, Tesseract OCR / Gemini Vision OCR	Parses business documents, extracts registration IDs, and reconciles claims against official certificates.

High-Level Dataflow Diagram



3. Database Architecture & Relational Schema

The persistence layer uses SQLAlchemy ORM mapping to PostgreSQL (or SQLite locally). The primary schema is modeled across four core entities:

Entity Relationship Breakdown

1. `schemes` Table

Holds structured government scheme definitions, criteria rules, financial subsidies, and document requirements.

Column	Type	Constraints	Description
`id`	UUID / Integer	PK, Autoincrement	Unique identifier for the scheme
`name`	String(255)	Indexed, Not Null	Official name (e.g. CGTMSE, PMEGP)
`short_name`	String(50)	Nullable	Common acronym or abbreviation
`ministry`	String(255)	Not Null	Governing ministry (e.g., Ministry of MSME)
`description`	Text	Not Null	Comprehensive description of benefits and scope
`funding_type`	String(50)	Not Null	Subsidy, Loan, Credit Guarantee, Grant
`max_benefit_amount`	Numeric / Float	Nullable	Maximum financial benefit in INR (₹)
`eligibility_rules`	JSON / Text	Not Null	Structured rules (turnover max/min, sectors, categories)
`required_documents`	JSON / Array	Not Null	List of required certificates (PAN, Aadhaar, Udyam)
`application_url`	String(500)	Nullable	Direct URL to official national/state portal
`is_active`	Boolean	Default True	Active state flag for recommendations

2. `business_profiles` Table

Stores the MSME business attributes used by the eligibility engine.

Column	Type	Constraints	Description
`id`	UUID / Integer	PK	Unique profile identifier
`user_id`	String(255)	Indexed, Unique	Supabase Auth User UUID
`business_name`	String(255)	Not Null	Registered enterprise name
`enterprise_type`	String(50)	Not Null	Micro, Small, Medium
`sector`	String(100)	Not Null	Manufacturing, Service, Trading
`annual_turnover`	Numeric / Float	Not Null	Reported annual turnover in INR (₹)
`investment_plant_machinery`	Numeric / Float	Not Null	Plant, machinery or equipment investment (₹)
`state`	String(100)	Not Null	State or Union Territory of operation
`district`	String(100)	Nullable	District name
`gender`	String(20)	Not Null	Male, Female, Other (for women schemes)
`social_category`	String(50)	Not Null	General, OBC, SC, ST, Minority
`has_udyam`	Boolean	Default False	Whether enterprise has Udyam registration
`udyam_number`	String(50)	Nullable	Udyam Registration Number (UDYAM-XX-00-0000000)

3. `recommendations` Table

Logs calculated matches, scores, and Explainable AI rationales for each business.

Column	Type	Description
`id`	UUID / Integer	Primary Key
`business_profile_id`	FK	References `business_profiles.id`
`scheme_id`	FK	References `schemes.id`
`match_score`	Float (0 - 100)	Calculated compatibility percentage
`status`	String(50)	Eligible, Partially Eligible, Disqualified
`reasons`	JSON	Array of specific qualification or disqualification points

4. Backend Deep Dive (FastAPI Services & APIs)

Directory Architecture (`backend/app/`)

```
backend/
├── app/
│   ├── api/
│   │   ├── admin.py            # PDF ingestion, scheme curation, Qdrant indexing
│   │   ├── auth.py             # User profile endpoints (/me)
│   │   ├── business.py         # Business profile management CRUD
│   │   ├── ocr.py              # Document upload & optical character recognition
│   │   ├── recommendations.py  # Evaluation & matching engine endpoints
│   │   └── schemes.py          # Scheme listing, details, and RAG chat endpoint
│   ├── services/
│   │   ├── ai_search_service.py      # Hybrid keyword & semantic search
│   │   ├── auth_service.py           # JWT verification with optional guest support
│   │   ├── eligibility_engine.py     # Deterministic MSME rule validation
│   │   ├── embedding_service.py      # Google Generative AI embeddings generator
│   │   ├── ingestion_pipeline.py     # Document text chunking & vector storage
│   │   ├── ocr_service.py            # Document parsing & text extraction
│   │   ├── rag_service.py            # RAG controller with Gemini & DB fallback
│   │   ├── recommendation_engine.py  # Orchestration & scoring pipeline
│   │   └── storage_service.py        # Object storage (Supabase buckets)
│   ├── config.py           # Pydantic Settings & environment loader
│   ├── database.py         # Engine setup & session dependency injection
│   ├── models.py           # SQLAlchemy declarative database models
│   └── schemas.py          # Pydantic request/response schemas
├── run.py                  # Uvicorn server launcher (port 8000)
└── requirements.txt        # Python dependency manifest
```

Key API Endpoints Reference

Method	Endpoint	Auth Required	Description
GET	`/health`	No	Server heartbeat check and system status
GET	`/api/v1/auth/me`	Yes	Fetches current user metadata and active business profile
POST	`/api/v1/business/profile`	Yes	Creates or updates the MSME business profile
GET	`/api/v1/business/profile`	Yes	Retrieves the authenticated user's business profile
GET	`/api/v1/schemes/`	No	Returns list of all active schemes with filtering options
GET	`/api/v1/schemes/{id}`	No	Fetches complete scheme dossier including eligibility rules
POST	`/api/v1/schemes/chat`	**Optional**	RAG-powered conversational scheme assistant (Gemini LLM)
GET	`/api/v1/recommendations/`	Yes	Runs matching engine and returns eligible schemes with XAI scores
POST	`/api/v1/ocr/upload`	Yes	Accepts PDF/Images, runs OCR extraction, and validates Udyam/PAN
POST	`/api/v1/admin/ingest-pdf`	Yes (Admin)	Parses scheme circular, chunks text, and vectorizes into Qdrant

Eligibility & Rules Evaluation Engine (`eligibility_engine.py`)

The eligibility engine evaluates deterministic rules defined in JSON against the user's `BusinessProfile`. Criteria checks include:

- **Enterprise Classification Check:** Validates against Government of India MSME criteria:
  - *Micro:* Investment ≤ ₹1 Crore AND Turnover ≤ ₹5 Crore
  - *Small:* Investment ≤ ₹10 Crore AND Turnover ≤ ₹50 Crore
  - *Medium:* Investment ≤ ₹50 Crore AND Turnover ≤ ₹250 Crore
- **Turnover Limits:** Checks `min_turnover` and `max_turnover` thresholds.
- **Sector Compatibility:** Checks if scheme targets Manufacturing, Service, or Trading.
- **Demographic & Diversity Boosters:** Validates criteria for Women Entrepreneurs, SC/ST, Minority, or North Eastern Region (NER) enterprises.
- **Registration Prerequisites:** Enforces whether Udyam or GST is strictly mandated.

RAG Service & AI Chatbot Pipeline (`rag_service.py`)

The Retrieval-Augmented Generation (RAG) service enables natural language interaction with policy circulars.

Vector Semantic Search (Primary)

1. Converts query to a 768-dimensional vector via Google text-embedding model.

2. Queries Qdrant collection `msme_schemes` using cosine similarity.

3. Retrieves top-k relevant paragraph chunks from government guidelines.

SQL Database Fallback (Resilient)

1. Activated automatically if Qdrant is unreachable or returns 0 matches.

2. Extracts keywords from user query and runs ILIKE filters across all 213 local schemes.

3. Compiles ministry, max benefits, eligibility, and descriptions as context.

LLM Prompt Construction: The retrieved context is formatted into a system prompt for Google Gemini, instructing the model to provide accurate, grounded, bulleted responses with official scheme citations.

5. Frontend Deep Dive (React 19 + Vite Application)

Directory Architecture (`frontend/src/`)

```
frontend/
├── src/
│   ├── assets/             # Static logos and illustrations
│   ├── components/
│   │   ├── Chatbot.jsx       # Resilient AI Scheme Assistant widget
│   │   ├── Navbar.jsx        # Top navigation header with profile & auth actions
│   │   ├── ProtectedRoute.jsx# Auth gatekeeper with Supabase Login/Signup modal
│   │   ├── SchemeCard.jsx    # Scheme presentation card with match score & badges
│   │   └── Sidebar.jsx       # Side navigation bar for application views
│   ├── context/
│   │   └── AuthContext.jsx       # Global Supabase authentication & user state
│   ├── pages/
│   │   ├── AdminPanel.jsx          # Admin ingestion & vector index management
│   │   ├── Analytics.jsx           # Analytics charts for matched scheme distributions
│   │   ├── BusinessProfile.jsx     # Multi-step business profile questionnaire
│   │   ├── Dashboard.jsx           # Executive overview & stats dashboard
│   │   ├── DocumentVerification.jsx# OCR file upload and certificate reconciliation
│   │   └── RecommendationsList.jsx # Matched schemes with filters & XAI reasons
│   ├── utils/
│   │   └── api.js            # Configured Axios instance with bearer token interceptor
│   ├── App.jsx               # Tab-based routing controller and root layout
│   ├── index.css             # Tailwind CSS tokens and custom styling
│   └── main.jsx              # React DOM initialization
├── vite.config.js          # Vite config with /api reverse proxy
└── package.json            # Node.js dependencies manifest
```

Core Pages & Functionality

1. Dashboard (`Dashboard.jsx`)

Serves as the mission control for the entrepreneur. Displays profile completion percentage, count of eligible schemes, total potential subsidy unlock amount, and quick links to apply.

2. Business Profile (`BusinessProfile.jsx`)

An interactive, validated form capturing enterprise parameters (Annual Turnover, Plant/Machinery Investment, Sector, State, Social Category, Gender of Promoters, Udyam Registration). Updates trigger real-time re-evaluation of recommendations.

3. Recommendations Hub (`RecommendationsList.jsx`)

Displays all schemes scored by the recommendation engine. Filterable by category (Subsidy, Loan, Credit Guarantee). Each card expands to show Explainable AI (XAI) bullet points

justifying why the user qualifies or what is missing.

4. Document Verification (`DocumentVerification.jsx`)

File drop zone supporting PDF and image uploads of Udyam certificates, PAN cards, and GST filings. Displays OCR processing status and verified attributes extracted from the document.

5. Scheme Assistant Chatbot (`Chatbot.jsx`)

A floating, draggable widget available on every page. Users can ask open-ended questions ("What is CGTMSE?", "What are the rules for women entrepreneurs?"). Features quick prompt suggestions, typing indicators, auto-scrolling, and formatted markdown response rendering.

6. Data Ingestion & Dataset Tools

The repository includes standalone utilities for scraping, extracting, verifying, and seeding scheme data:

Script / File	Purpose
`extract_schemes.py`	Extracts structured scheme data from raw text sources and PDF circulars into standardized JSON format.
`extract_all_schemes.py`	Batch extraction script that processes the entire catalog of central and state MSME schemes into `extracted_schemes.json`.
`extracted_schemes.json`	Master database file containing 213+ fully structured government schemes with ministry, benefits, rules, and documents.
`sample_schemes.json`	Lightweight sample dataset used for integration testing and staging environments.
`check_schemes.py`	Validation script that inspects database integrity and verifies scheme counts and JSON schema compliance.
`check_qdrant.py`	Diagnostic script to test latency, collection status, and vector dimension consistency with Qdrant Cloud.
`start_all.bat`	One-click Windows batch script that starts both backend and frontend servers in separate command windows.

---

7. Recent Resiliency Fixes & Architectural Upgrades

> **Resilience Upgrades Implemented**
>
> To eliminate intermittent network drops and authentication rejections, five major architectural enhancements were made to the codebase.

1. Optional Authentication on Chatbot Endpoint

- **Prior Behavior:** `POST /api/v1/schemes/chat` required strict authentication via `get_current_user`. If a user was browsing as a guest, had an expired token, or token refresh failed, the backend returned `401 Unauthorized`, causing the frontend to report a network failure.
- **Solution:** Added `get_optional_current_user` in `auth_service.py` with `HTTPBearer(auto_error=False)`. The chatbot endpoint now accepts both authenticated users and guests seamlessly.

2. Automatic SQL Database Fallback for Vector Search

- **Prior Behavior:** When remote Qdrant Cloud connections experienced socket drops (`[WinError 10054] ConnectionResetError`), vector search threw an unhandled exception, causing the chatbot to fail.
- **Solution:** Implemented a database fallback mechanism in `rag_service.py` . If vector search fails or returns zero hits, the service queries the local SQL database across all 213 schemes using tokenized keyword search. This context is injected into Gemini, ensuring reliable answers even when offline.

3. Frontend Vite Reverse Proxy Configuration

- **Prior Behavior:** Axios called `http://localhost:8000/api/v1` directly. When frontend was accessed via `127.0.0.1` or different ports, CORS preflight checks failed intermittently.
- **Solution:** Configured a reverse proxy in `vite.config.js` forwarding `/api` requests to `http://localhost:8000` , eliminating CORS issues during development.

4. Improved Chatbot Error Handling & Formatting

- **Prior Behavior:** Frontend caught errors and displayed generic `Error: Network Error` with unformatted plain text.
- **Solution:** Added explicit network error detection in `Chatbot.jsx` with actionable guidance, and added `whitespace-pre-wrap` styling for proper markdown rendering of lists and paragraphs.

5. Robust Windows Startup Script

- **Prior Behavior:** `start_all.bat` used relative `cd` commands that failed if executed from another working directory or via desktop shortcuts.
- **Solution:** Updated paths to use `cd /d "%~dp0backend"` and `cd /d "%~dp0frontend"` , guaranteeing correct directory resolution.

8. Setup, Execution & Operations Guide

Prerequisites

- Python 3.10+ installed and added to PATH
- Node.js 18+ and npm installed
- A Google Gemini API key (configured in `backend/.env`)
- (Optional) Supabase project credentials and Qdrant Cloud cluster

Quick Start (Windows)

Double-click the `start_all.bat` file in the repository root. This will automatically:

1. Open a command window and start the FastAPI backend on `http://localhost:8000`
2. Open a second command window and start the Vite development server on `http://localhost:5173`

Manual Start Commands

Backend

```
cd backend
# Activate virtual environment if present:
.venv\Scripts\activate
# Install dependencies:
pip install -r requirements.txt
# Launch development server:
python run.py
```

Frontend

```
cd frontend
# Install dependencies:
npm install
# Launch development server:
npm run dev
```

Access URLs

Service	URL	Purpose
Frontend Application	`http://localhost:5173`	Main web application user interface
FastAPI Swagger UI	`http://localhost:8000/docs`	Interactive OpenAPI specification and API testing sandbox
FastAPI ReDoc	`http://localhost:8000/redoc`	Alternative technical API documentation

© 2026 Vyapar SchemeHub Platform • Built for India's MSME Ecosystem • End of Comprehensive Documentation